

Flash Gordon Development Guide

Last updated Jan 11, 2024.

Written by Daniel Terry and Manuel Juette

1 INTRODUCTION

Flash Gordon is an instrument control software package for widefield single-molecule fluorescence measurements, especially smFRET. This document provides an overview of the code base and tips for adding new features. Before making any changes, please review the [LabVIEW programming paradigms](#).

The architecture is based on the LabVIEW [Actor Framework](#) and aims to provide enough abstraction and modularity to allow for relatively easy adaptation to different setups. This guide explains specific design features and development workflows for Flash Gordon. It is not a general introduction to the Actor Framework or LabVIEW. A good starting point to learn the basics of the actor framework is the *Actor Framework Fundamentals* project from the Example Finder, along with documents found at <https://goo.gl/A86KuP>.

Please read “Flash Gordon Documentation.pdf” first before this document to review the normal functioning of the program.

2 PREREQUISITES

Prerequisites for running the *compiled* Flash Gordon executable are listed in “Flash Gordon Documentation.pdf”. For *development*, the following additional requirements need to be met:

All libraries supported by Flash Gordon must be installed to avoid link errors, even if the developer doesn’t plan to use them. It is possible to bypass this restriction by removing a device lvlib from the project and ensuring that it is never directly referenced in the source code.

- 64-bit LabVIEW Professional version 2023 Q3 (newer versions may also work but are untested). This requires a license to be purchased. <https://www.ni.com/en-us/shop/labview.html>
 - DAQ-mx (for the USB-6501 device):
<https://www.ni.com/en-us/support/downloads/drivers/download.ni-dagmx.html#445931>
- Latest version of DCAM (for Hamamatsu cameras). We have tested most extensively with versions 22.2.6391 and 22.8.6486. <https://dcam-api.com/>
- Install latest version of Hamamatsu Video Capture library for LabVIEW. This will install VIs into LabVIEW user.lib folder and install tmcamcon.dll to C:\Windows\System32.
<https://dcam-api.com/hamamatsu-software/>
- Andor SDK2 (for EMCCD cameras): tested with version 2.104.30084.0.
<https://andor.oxinst.com/downloads/uploads/Software%20Development%20Kit.pdf>
<https://andor.oxinst.com/downloads/view/andor-sdk-2.104.30084.0>

- Photometrics PVCAM and LabVIEW adapter. Flash Gordon depends on the current BETA version of the LabVIEW adapter (2.3.x), which is not publicly available. Please directly contact Photometrics to request the latest version.
<https://www.photometrics.com/products/pvcam>
<https://www.photometrics.com/support/third-party-software/labview>
- Nikon Ti2 SDK 64-bit, which includes the device driver for the Ti2 microscope.
<https://nisdk.recollective.com/microscopes>
- Thorlabs Kinesis SDK for LabVIEW.
https://www.thorlabs.com/software_pages/ViewSoftwarePage.cfm?Code=Motion_Control
- ASI stage controller driver for LabVIEW.
<https://www.asiimaging.com/support/downloads/ms-2000-control-using-nis-labview-and-the-serial-port/>
- Coherent Genesis driver. Your laser should have included a CD that will install an application called “OPSL” and the driver. If you need to install the driver directly, it can be found in a directory similar to “CDM v2.12.28 WHQL Certified” on the CD.
<https://www.coherent.com/support>
- Optional: Visual Studio with C/C++ tool chain (free Community Edition is sufficient) to recompile the C DLLs (specifically dcimg2tiff.dll) used for time-critical operations in Flash Gordon.
- Optional: MATLAB version 2018 or newer (required for the particle counting in the Live Viewer window).
- Optional: instructions for installing software for configuring the Arduino are list on the developer’s website: https://github.com/ximeg/sync_device

Once these packages are properly installed, deploy the Flash Gordon source code to “C:\FlashGordon\Development”. To run Flash Gordon, open the project file, double click on “Flash Gordon.vi” from within the LabVIEW project, and click the “play” button to start the program. **Do not start “Flash Gordon.vi” from outside the project!**

3 STRUCTURE

3.1 SUBSYSTEMS

Flash Gordon is divided into the following four subsystems:

1. **Application layer.** This Actor is the core system that controls the lifecycle of all the other Actors and acts as an abstraction layer between the hardware abstraction layer and the user interface elements. The Application actor maintains the state of the program.
2. **User Interface (UI) layer.** The *Main Window* and *Viewer* actors as well as actors for dialog boxes, progress windows etc. Together, these actors handle all user interactions. The UI elements communicate via messages only to the *Application* Actor and do not interact directly with the hardware layer.

3. **Hardware abstraction layer (HAL).** Provides generic device Actors (*Camera*, *Microscope*, etc.), each of which provides an interface (in the form of Message classes) that is common to all devices of that type. These Actors can receive messages from the *Application* actor and route them to the appropriate device layer actors corresponding to the initialized hardware. This approach minimizes coupling and ensures new devices can be easily added with minimal changes to the code base.
4. **Device layer.** Provides actors for all supported hardware devices. The methods override those in the HAL to include device-specific API calls. In some cases, the HAL and Device layers are merged, with specific devices handled by Case Structures within each method. This approach minimizes development time but is only appropriate for simple devices.

3.2 CLASS HIERARCHY

The main classes within Flash Gordon are actors, which are derived from the Actor Frameworks *Actor* class. The core *Application* actor implements the Application subsystem. *UI Window* and *Device* are abstract classes that provide generic functionality for the UI and Hardware subsystems; specific windows and devices are implemented by subclassing. Messages between actors are derived from a separate *Message* class, which is also part of the framework.

Actor

- Application
- TIFF Writer
- UI Window
 - Main Window
 - Viewer
- Device
 - Camera
 1. Hamamatsu
 2. Andor
 3. Photometrics
 - Laser
 1. LaserQuantum
 2. Coherent
 3. Thorlabs (polarizer)
 - Microscope
 1. Nikon Stand
 - Power Meter
 - Stage
 - Sync Device
 1. NI DAQ
 2. Arduino DAQ

3.3 CONFIGURATION FILES

Flash Gordon uses two types of configuration files. Both are XML files based on LabVIEW's own schema and type-def'd clusters. This makes them easy to generate using VIs, while also being reasonably human-readable and editable. *Init.xml* in the main project directory contains general settings, while the *Config* directory can contain any number of hardware configuration files that define camera channels, COM ports and other specifics of the available devices.

4 WALKTHROUGH

Always open the project file (.lvproj) in the Flash Gordon directory to examine the code or make any changes. Avoid running or modifying files outside the project.

The main subsystems are contained in virtual folders, including *Application*, *Hardware*, and *User Interface* described above. The *Resources* virtual folder contains type defs, global variables, and various VIs used for development/testing.

Within the virtual folders for the different subsystems, each actor is contained in a separate LabVIEW library (.lvlib) file. This includes:

1. A virtual folder with the message classes for this actor.
2. The actor class itself. This, in turn, contains the private data cluster and separate virtual folders for protected and public methods, getters and setters. Notes:
 - i. LabVIEW does not enforce any particular organization of class members. Adhering to a fixed virtual folder structure prevents confusion.
 - ii. Each public method corresponds to a message class and vice versa, as this is the framework's mechanism to expose functionality to calling actors.

At the top level, there are only three VIs:

- 1 *Flash Gordon.vi* – This is a minimal launcher for the top-level *Application* actor, which handles all other tasks.
- 2 *Create Initialization File.vi* and *Create Hardware Configuration.vi* – As their names imply, these VIs provide a simple way to write config files.

4.1 GENERAL PROGRAM FLOW

This section describes the general execution flow for a typical workflow.

- 1) *Flash Gordon.vi* launches the *Application* Actor.
- 2) *Application/Actor Core.vi* launches the *Splash Screen* and selects the hardware configuration using the "Load Configuration.vi" dialog, saving this information in the *Application* class data.
- 3) *Application/Init Hardware.vi* launches all of the various device actors, as dictated by the selected configuration file. The *Splash Screen* updates as each device completes its initialization procedure.

- 4) Once the hardware is ready, the *Splash Screen* is closed and the *Main Window* actor is launched. Some controls in the Main Window are populated according to the hardware configuration passed from the application, but are not updated if these are later changed.
- 5) Main Window/Actor Core.vi contains an event loops with a central Event Structure with cases for every control the user may interact with. User interaction triggers an event case, in which a message is sent to the *Application*, which responds generally by forwarding this message to device actors. For example, pressing “Stage Forward” sends a Stage Forward message to the *Application*, which is then forwarded to the *Stage* object.
- 6) When device actors send status update messages to the *Application*, they are forwarded to the *Main Window* class, which passes this information to the Actor Core.vi via a User Event and associated Event Case. For example, after manual stage repositioning, the *Stage* actor will send the current stage position to be displayed as a red dot in the *Main Window*.
- 7) The state of the *Main Window* is always dictated by the state of the *Application*, rather than changing state directly as a result of user actions. When an acquisition starts, some controls are disabled, and these are re-enabled when this task completes. This process controlled by “Application Mode Event” messages.
- 8) “Stream Acquisition” triggers a more complex procedure:
 - a. Request *Camera* prepare for acquisition (synchronous), wait for *Countdown Timer* (if applicable), show the *Viewer* window, and direct the *Synchronization Device* actor to send TTL pulses on control lines to trigger cameras, shutters, fluidics, etc.
 - b. During acquisition, the *Camera* actor reads acquired data and saves it to global variables “Live Frame” (image data) and “Current Frame” (frame number). The *Viewer* actor polls these variables to display the current state of the acquisition.
 - c. When the *Camera* actor sees that the acquisition is complete, it sends a “Finish Stream” message to the *Application*, which responds by closing the *Viewer* window, saving the acquired image data (via the *Tiff Writer* actor), and sending a message to the *Main Window* that returns it back to the default ready state.
- 9) When the Main Window is closed by the user, it sends a stop signal to the *Application* actor. No specific shutdown code is required within the *Application* actor, as all nested actors are set to “auto-stop” together with their calling actor (at which point they all execute their respective termination code). If an actor does not respond to the shutdown message (such as if a message handler method got stuck in an infinite loop), the program may continue executing the background and must be stopped with Task Manager.

4.2 AUTOMATION

A series of movies can be acquired, with a sequence of events for each that may include stage movement, laser power adjustment, image-based autofocus (from a Z stack), and stream acquisition that is repeated for each movie. This process is handled by Application/Actor Core.vi. The automation series is started by a “Start Recording” User Event that triggers an Event Case within Application/Actor

Core.vi. The implementation of these automated movie series is not implemented in the best way and is likely to change in the future.

5 CREATING A NEW ACTOR CLASS

5.1 CREATING A NEW *DEVICE* ACTOR

Below is the general process for creating a new *Device* actor.

Creating a new device class:

- Create a new class inheriting from the *Device* class, or a more specific class that defines a specific type such as *Camera*, *Laser*, or *Sync Device*.
- Add any device-specific data to the properties in the .ctl file associated with the class.
- Create a public “Initialize.vi” method that takes an uninitialized instance (default value) of the class and uses its inputs to set class properties and prepare for execution. Any startup tasks that take time (such as connecting to a device) should be done in this vi. This VI is synchronous, so execution will stall until it completes.
- Create an override for “Shut Down Device” for any steps to cleanly shut down the actor, including shutting down the device and closing any device handles.
- Create override methods for any other methods as needed. For example, classes inheriting from *Camera* will override “Start Live.vi”, “Stop Live.vi”, and “Get Most Recent Frame (if active).vi” for viewing frame data.
- For classes inheriting from *Camera*, most methods should call the parent method to set variables that reflect the state of the device that are common to all devices of this type.
- For top level devices inheriting directly from *Device*, create messages for each of the public methods (right click and choose Actor Framework->Create Message).
- For devices from derived classes such as *Camera* or *Sync Device*, dynamic dispatch will be used to direct a message defined in the parent class to the specific class method of the actual device class. As such, these classes do not need to define any messages.

Supporting a new device type:

- For top-level devices directly inheriting from *Device*, call the device’s Initialize.vi from within the “Init Hardware.vi” or another sub-vi it calls and use “Launch Actor.vi” to start the actor.
- For derived devices (*Camera* or *Sync Device*), simply add a new case to the case structure that is wired into “Launch Actor.vi”

- If applicable, update the typedef listing all of the devices of this type to add the new one.
-

Note about using the *Pre Launch Init* method: actors CANNOT be launched there (attempting this will cause the program to hang without any error message). *Pre Launch Init* should only be used to check if pre-requisites for running the core are met (e.g. parameters initialized?) and abort launching the actor if necessary.

5.2 CREATING A NEW USER INTERFACE WINDOW ACTOR

User interface windows that need to run continuously and asynchronously from other actors should be instances of the *UI Window* class. For many modal dialog type use cases, it is best to use a simple VI wired directly into the calling VI with the front panel visible. The VI will block execution until the task is complete. For an example of this use case, see “Countdown Timer.vi”.

Here are the basic steps for creating a new UI Window class:

- Create a new class inheriting from UI Window
- Create an override of the “Create Front Panel Events” that calls “Create User Event.vi” for each type of message that may need to be passed to Actor Core. Bundle these into the class data and update the class .ctl file to include them.
- Create an override for “Destroy Front Panel Events” that calls “Destroy User Event.vi” to clean up all of the User Event instances.
- Create an override method for Actor Core.vi. This will generally include all user interface elements on the front panel and a helper loop with an Event Structure in the wire diagram. User interactions with front panel controls trigger cases in the Event Structure and perform the desired actions. Be sure to wire the various User Event properties from the class data into the Event Structure so Actor Core can receive these events.

IMPORTANT: Don’t wire TRUE to “Show Front Panel” option when launching the actor, as this will not work for built application. Use VI properties of Actor Core.vi file instead.

5.3 CREATING A NEW METHOD

First decide whether the method should be public (often used for message handling) or private (internal methods with no associated messages). To override the behavior of a parent class, right click on the virtual folder associated with this scope and select “Add->VI for Override”. To add a method that is specific to the class, right-click on the fold and select “Add->VI from dynamic dispatch template”.

Utility methods that do not use class data should be created as an ordinary VI and added to the .lvlib outside of the class hierarchy.

5.4 ADDING AND ACCESSING CLASS PROPERTIES

Class properties are defined in a .ctl file in each class, to which new properties can be directly added. These are easily accessed by class methods using the cluster “bundle/unbundle by name” Vis wired form

the class instance wire. To access properties inherited from the base class (such as *Device* or *Camera*), either use property nodes or create a new method in the parent class specifically designed for this task.

5.5 CREATING A MESSAGE

Actors generally communicate by asynchronous message passing. This approach minimizes coupling between actors and allows them to run simultaneously and independently with minimal chances of race conditions.

Messages are processed by the actor in a first-in-first-out order (not in parallel). As such, any method used to respond to a *Message* should generally execute in minimal time; tasks that may block while waiting for a resource to become available or are computationally intensive should not be executed directly in message handling methods. These are best processed within Actor Core.vi (or passed along to some other actor specialized for this task).

To create a new *Message* for an existing method, first make sure the target vi is in the “Public” scope and its connector pane has been finalized, including which inputs are required, as this will be harder to change later. Next, right click on the desired method vi and select “Actor Framework->Create Message”. This will execute a script to create a new message class, which should reside in the “Messages for this actor” virtual folder. The class properties include all elements for each of the VI inputs. The methods include a Do.vi that calls the target method and a “Send <method name>.vi” that is used to send a message to the target actor. None of these files should be edited directly by the developer.

It is possible to send messages synchronously by inheriting from the “Send Message And Wait For Response” VI, but this is not recommended as it can create race conditions and was intentionally made by NI to be difficult to implement. Currently, this paradigm is only used by “Start Autofocus Msg”.

5.6 EDITING A MESSAGE

Changes to class methods do not generally require any changes to the associated message class. However, if you have changed the connector pane of a method VI, including which terminals are required, you need to update the associated message class. This is accomplished by right-clicking on the message class and choosing “Actor Framework->Rescript message”.

5.7 GLOBAL VARIABLES

Whenever feasible, all communication between actors should be done by passing messages, but there are some exceptions.

For performance and memory usage reasons, it is not recommended to pass large blocks of data in a *Message* (such as image data). Instead, use a global variable in “Resources/GLOBAL Time-Critical.vi” or pass a Data Value Reference (DVR). Extra care must be taken in these instances to ensure that the data flow is unidirectional with a single writer, and the data is protected from race conditions.

Some status variables are communicated synchronously, such as “Trigger Ready” and “Current Frame”. These are stored “Resources/GLOBAL Time-Critical.vi” to bypass the overhead associated with message passing and simplify the implementation. This should be avoided when possible and is likely to change in the future.

Another global variable (“GLOBAL Experiment Metadata.vi”) is used to store device status and acquisition settings to be used for saving metadata to movie files. This is done to simplify the implementation to avoid creating a large number of new messages but is likely to be removed in future versions.

5.8 ACTOR CORE.VI

Rather than running in response to a specific, Actor Core runs continuously throughout the lifetime of the actor. Actor Core typically splits the actor instance wire to a “message handler” loop (implemented by the Actor Framework) and a [“helper loop”](#) (in the override method). The helper loop runs in its own thread separate from message handling, which creates several issues that the developer should keep in mind:

- The ‘stale’ actor object wire is not updated when message handling methods modify the class data over time. This wire is still useful, for example for referencing the device configuration, which cannot be changed in Flash Gordon after initialization.
- Class methods should not be called directly from Actor Core using the stale actor object wire. Instead, use “Read Caller Enqueuer.vi” to self-enqueue messages, which will call class methods using the active reference to the class data.
- Class methods can communicate with the Actor Core helper loop in a few ways:
 - a. **User Events:** “Create User Event” VI creates a reference to a queue that is saved in class properties. Class methods call the “Generate User Event” VI to add an event to the queue. Actor Core wires a reference to the queue to its Event Structure and add cases to react each user event type. The class destructor should call “Destroy User Event” to clean up these references. This approach is ideal for reactive actors such as *UI Window* instances.
 - b. **Data Value Reference (DVR):** a DVR can be referenced by both class methods and Actor Core to refer to the same memory location. This approach is most valuable for time-critical applications such as in *Synchronization Device*. The flow of information should be unidirectional and extra care must be taken to avoid race conditions.
 - c. **Global variables:** this approach should be avoided if possible. It is currently used in Flash Gordon for broadcasting some information across the whole application, such as the current frame number. This behavior will be changed in a future version.
 - d. Notifiers, queues, and some other objects are passed by reference and therefore can be used for communication between Actor Core and other class methods, but this isn’t standard in Flash Gordon. This can include references to front panel controls, which can be useful for real-time updates of the user interface elements.

6 ADDING SUPPORT FOR NEW DEVICES

This section describes how to add support for new devices to the source code. In most cases, a new case must be added to each method within a class. But for some more complex classes, specifically the Camera class, a new inherited class must be created.

Sync Device:

- Many National Instruments DAQ devices are likely supported by simply modifying the NI-DAQmx physical channel addresses in the configuration file.
- Other Arduino-type devices can be supported if they can be programmed to accept the expected commands. The protocol is described here: https://github.com/ximeg/sync_device

Shutters: any shutter driver that can be triggered by TTL (0/+5V) logic should be supported by Flash Gordon. Other optical modulators, such as AOM, EOM, and AOTF devices, can be similarly controlled if they can be programmed to accept TTL signals from the DAQ.

Lasers: support for controlling laser power directly at the laser controller or via some attenuator device (such as a motorized rotating polarizer) can be added by modifying the following files in “Hardware\Laser\”. Each of these functions includes a case structure with a case for each supported device type.

- **Init Device.vi:** establish a connection with the device and perform any startup tasks. For RS232 (serial) connections, this is where baud rate and other parameters are set.
- **Set Laser Power.vi:** send commands to the device to adjust laser power.
- **Shut Down Device.vi:** send any shut down commands and close the connection.

Power meter: support for new power meters can be added by adding a case to each of the files in “Hardware\Power Meter” and updating the associated typedef and config files.

Stage: to add support for a new stage, modify files in “Hardware\Stage Controller\”. Each of these has a Case Structure for each of the supported stages. Be aware that some stages use the same API as the microscope stand, so there may be overlap (see below).

Microscope: to add support for a new microscope stand, and specifically the capability to change lightpath direction and filter set, modify the following files.

- **Resources\Type Defs\Microscope Type.cti:** add a value for the new API.
- **Hardware\Microscope\Init Device.vi:** launch a new Actor that you will create.
- **Hardware\Microscope\Set Detection.vi:** send messages to the new Actor to direct the microscope to change the lightpath and filter positions.
- **Config file:** The integer values for light path and filter selection are meant to be generic and will depend on the specific API.

Hamamatsu cameras:

- **Hardware\Hamamatsu Camera\Max Lines.vi:** This function calculates the optimal readout mode and maximum number of camera lines that can be read at the target frame integration time. Currently this assumes 2x2 binning and USB connection, which simplifies the calculation.

To add support for a new camera, add a new case corresponding to the camera model string. Read the camera manual to determine the parameters and verify the results with an oscilloscope. Incorrect parameters may result in skipped frames or inconsistent timings.

- **Hardware\Hamamatsu Camera\Prepare Acquisition.vi:** This function might need modifications if some standard features are not supported.
- **NOTE:** the Hamamatsu Lightning (C14120-20P) has unusual timing properties that make it harder to support and may require more significant adjustments to the architecture.

Cameras from a new vendor/API:

- Create a new actor modeled after the Hamamatsu and Photometrics actors, inherited from the Camera class.
- **Resources\Type Defs\Camera Type.ctl:** add a new enum value for the new vendor/API.
- **Config files:** update the Camera Type enum in all entries.
- **Application\Initialize Virtual Camera.vi:** Add a case to the case structure to launch the new Camera Actor.
- **DLLs:** drop any necessary libraries into the root folder with the FlashGordon.exe file, or into the “dll” folder if directly specifying the path.
- **BinaryTIFF:** depending on the implementation, you may need to add a new function to the dcimg2tiff library and new case to “Merge DCIMG Files.vi” to call it. This will be used to transform the raw data saved to disk by the camera driver into a tiff file.

Triggering other devices: to trigger other devices during acquisition using TTL logic (0/+5V), such as fluidics systems or optical modulators (EOM, AOM, AOTF), modify the code in “Hardware\Synchronization Device\”, particularly sync_alex_iteration.vi.

Configuration files: any changes to typedefs will cause problems when loading configuration XML files saved with another definition of the typedef. This is because LabVIEW’s XML schema includes a full definition of the typedef and a list of all possible values in any enum. To avoid this problem, use “Create Hardware Configuration.vi” or modify the files in a text editor. A new tool was recently created to migrate older versions, but this tool is not yet stable.

7 OTHER NOTES AND GUIDELINES

7.1 COMPILING AN EXECUTABLE

Except for testing and development, Flash Gordon should always be run as a compiled binary. To compile the executable, follow these steps. Version numbers are specified as MAJOR.MINOR.BUGFIX. Major versions break significant functionality users may depend on (such as migrating the new file formats). Minor versions implement significant new features. Bugfixes are slight changes to the code that only correct a specific bug and add no new features.

- 1) Update the version number in the Init.xml file using a text editor.
- 2) In the project, double-click on “Build Specifications->Flash Gordon”.
- 3) Click on the “Version Information” tab and update the version number. We use the current date as the build number (e.g., 1023 for Oct. 23rd).

- 4) If new files are required that LabVIEW may not find by traversing the VI dependencies (such as image and dll files or classes referenced only by file path), these can be added in the “Source Files” page. This also includes any classes that are dynamically loaded at runtime via their file path, which is used for many device actors so they are not loaded unless needed.
- 5) Click “Build” to compile the project.
- 6) The destination for the compiled project should be “C:\Flash Gordon\testing” for unstable versions to be tested and “C:\Flash Gordon\master” for stable releases deployed to production workstations.

Keep in mind the executable will need any dll files it depends on for loading. The Thorlabs libraries depend on .NET, which we have found requires them to be in the same folder as Flash Gordon.exe. Other dependencies are saved in the “dll” folder.

7.2 BINARYTIFF

This is a standalone LabVIEW/C++ project used to combine raw files streamed from the camera library to disk into TIFF files, where the different spectral channels are tiled side-by-side (up to four channels in a 2x2 grid), after any image flipping if requested. We use this approach to allow the device driver to stream directly to disk to avoid dropped frames. For Hamamatsu cameras, the utility reads .dcimg files using the DCIMGAPI library. For Photometrics, raw pixel data are saved that can be read directly. For further documentation, see the project directory.

7.3 TEAM DEVELOPMENT

Due to LabVIEW’s graphical nature and binary source code, version control has some unique challenges but is doable. Below are some tips to make the process as painless as possible:

- 1) Only commit files that were intentionally changed to limit scope. LabVIEW has a tendency to alter files that the user did not touch for obscure reasons. For example, great care should be taken in committing the main project file to avoid conflicts. This should not generally be necessary unless a completely new file has been included.
- 2) Never commit the “.aliases” and “.lvpls” files, which are workstation specific. Use .gitignore if possible to exclude them explicitly.
- 3) Although possible, merging multiple commits to a single file isn’t recommended. If multiple contributors are working on the code base at the same time, take extra effort to minimize the scope of changes to avoid possible conflicts. If larger scale changes are needed (such as to widely-used typedefs), communication is essential to coordinate commits without conflicts.
- 4) It is possible to diff or merge commits using LVDIFF and LVMERGE utilities, but requires some work to set up and is finicky. https://labviewwiki.org/wiki/Set_up_differencing_capabilities
- 5) Changes to typedefs (including enums) will cascade through every file that uses the typedef, which increases the likelihood of conflicts. Avoid changes to typedefs when possible.

- 6) Review this article about separating source and compiled code. If this option is not selected, source files are more likely to be altered indirectly and cause issues for version control.
<https://www.ni.com/docs/en-US/bundle/labview/page/facilitating-source-control-by-separating-compiled-code-from-vis-and-other-file-types.html>